

1. Architecture & Design Hotspots Analysis

Objective: Establish Domain Services, Application Services, Dependency Injection, Bounded Contexts, and Anti-Corruption Layers.

Date: July 15, 2026 | **Scope:** `/home/shweta.shende/Shweta/Code/multi-agent-web-ui` — React/Vite frontend + Node.js/Express backend + MongoDB

Executive Summary

EXECUTIVE SUMMARY

This multi-agent web application demonstrates a mixed architectural maturity with significant hotspots requiring attention. The most severe issues are found in oversized frontend components (3,220 LOC useAppStore), fat controllers with excessive responsibilities (434 LOC integrationController with 26 methods), and missing service layer abstractions. The dominant risk is change amplification — modifications in one area require cascading updates across multiple layers, making maintenance costly and error-prone.

64

CONTROLLERS / HANDLERS

13

MODELS / ENTITIES

18

SERVICE CLASSES FOUND

0

REPOSITORY CLASSES FOUND

OVERALL CODEBASE RATING — ARCHITECTURE & DESIGN

High Risk

Driven by High-Risk God Classes, Fat Controllers, and Missing Repository Pattern.

1.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	GOOD	MODERATE	HIGH RISK	Measured	Rating
H1	Fat Controllers	Avg LOC per controller	<150	150–300	>300	236 LOC	MODERATE
H2	Missing Service Layer	Controllers accessing repos/models	<10	10–20	>20	5	GOOD
H3	Missing Repository Pattern	Direct DB access points	<10	10–20	>20	38	HIGH RISK
H4	Circular Dependencies	Dependency cycles	0	1–3	>3	0	GOOD

H5	Shared Utility Abuse	Utility files w/ business logic	0	1–5	>5	12	HIGH RISK
H6	Direct SQL in Controllers	ORM compliance %	>90%	60–90%	<60%	95%	GOOD
H7	God Classes	Classes >1000 LOC	0	1–3	>3	10	HIGH RISK
H8	Domain Boundary Violations	Cross-domain access points	0	1–5	>5	1	GOOD
H9	Shared Database Coupling	Tables shared across domains	<10%	10–30%	>30%	15%	MODERATE
F1	Business Logic in Components	Avg LOC per component	<150	150–300	>300	647 LOC	HIGH RISK
F2	Missing Frontend Service/Data Layer	Components w/ inline API calls	<10	10–20	>20	1	GOOD
F3	God / Oversized Components	Components >400 LOC	0	1–3	>3	10	HIGH RISK
F4	Prop Drilling / Global State Abuse	Max prop-drilling depth	≤2	3–4	>4	5 levels	HIGH RISK
F5	Legacy / Inconsistent Component Patterns	Legacy-pattern components	0	1–10	>10	2	GOOD

1.4 Actions Required

Hotspot	Action	Rating	Priority
God Classes	Split useAppStore.jsx (3220 LOC) into domain-specific stores; break down Dashboard.jsx and AgentDetail.jsx into focused components	HIGH RISK	CRITICAL
Missing Repository Pattern	Create ITeamRepository, IUserRepository interfaces with Mongoose implementations; inject into services via DI	HIGH RISK	CRITICAL
Shared Utility Abuse	Extract business logic from src/lib/ files into domain services: AgentOrchestrationService, WorkflowPermissionService	HIGH RISK	CRITICAL
Business Logic in Components	Move validation and workflow logic from components to custom hooks and services	HIGH RISK	CRITICAL
God / Oversized Components	Decompose 10 components >400 LOC into single-responsibility components	HIGH RISK	CRITICAL
Prop Drilling / Global State Abuse	Split global store into domain contexts; introduce component composition patterns	HIGH RISK	HIGH

1.5 Expected Outcomes

- **Separation of Concerns:** Clear boundaries between UI, business logic, and data access layers enabling independent testing and development
- **Independent Module Evolution:** Domain services and repositories allow teams to modify authentication, workflow, and integration logic without cross-cutting impacts
- **Testability Improvement:** Extracted business logic becomes unit-testable without UI rendering or database dependencies through dependency injection
- **Performance Optimization:** Split global state reduces unnecessary re-renders and enables more granular component updates
- **Maintainability Enhancement:** Smaller, focused components and services reduce cognitive load and minimize regression risk during modifications